Imagine this `render` function is normally invoked when information about a user is retrieved. Without using the null soak operator, the `render` function will cause an error when a user property is missing:

```
user =
  name:
    first: 'Donot'
    last: 'Callme'

render user
# TypeError: cannot read property 'phone' of undefined
```

You don't want a `render` function to show an error to a user when data is missing. Instead, you want to *suppress* internal errors when rendering. Think about it: users don't want to read all your internal errors. Avoid showing errors by using the null soak:

```
render = (user) ->
  """
  <html>
  Home phone for #{user?.name?.first}: #
{user?.contact?.phone?.home}
  """

user = null
render user: null
# <html>
# Home phone for undefined: undefined
```

To display something other than `undefined`, you can use the default operator `||` to specify a default value:

```
user = null
contact = user?.contact?.phone?.home || 'Not provided'
contact
# Not provided
```

This allows you to present the information you need without an explicit conditional.

Null soak is not for assignment

Because null soak is so convenient, it's tempting to use it with assignment: